Sequence Models

COMP4153 Natural Language Processing

Kian Ming Lim

School of Computer Science
University of Nottingham Ningbo China

Where we are in the module

Last lecture

- perceptrons and multilayer networks;
- forward pass, loss and backpropagation;
- sentence classification with an MLP;
- limitation: averaging loses word order.

Today

- represent a changing context over time;
- build and train a recurrent neural network;
- understand vanishing and exploding gradients;
- use LSTM and GRU gates to control memory.



Learning outcomes

By the end of this lecture, you should be able to:

- Explain how RNNs capture word order and maintain context through hidden states;
- Compute an RNN forward pass and describe its applications in NLP;
- Explain BPTT, vanishing and exploding gradients, and gradient clipping;
- Describe how LSTM and GRU gates work, and compare recurrent architectures for different NLP tasks.

Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

Why an averaged embedding is insufficient

Sentence A

The student answered the teacher.

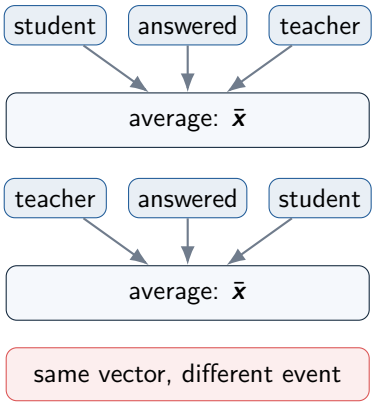
Sentence B

The teacher answered the student.

If a model only averages the same word embeddings:

$$\bar{\boldsymbol{x}} = \frac{1}{T} \sum_{t=1}^{T} \boldsymbol{x}_t$$

the two representations are identical.



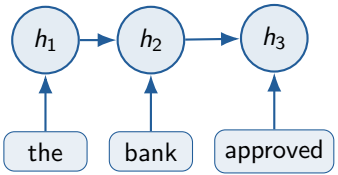
Words are not independent

Context changes interpretation

- *The bank approved the loan.*
- *They sat on the river bank.*

Order changes meaning

- *The student answered the teacher.*
- *The teacher answered the student.*



each state combines
the current word and earlier context

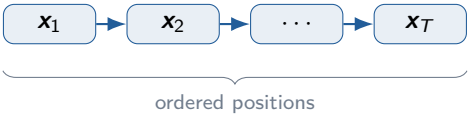
Key idea: A sequence model computes a representation whose meaning depends on both **what appears** and **where it appears**.

What is a sequence?

A sequence is an ordered collection:

$$\boldsymbol{x}_{1:T} = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T)$$

- T: sequence length;
- $\boldsymbol{x}_t$: representation at position t;
- order is part of the input;
- different examples may have different lengths.



Examples

Words in a sentence, characters in a word, sentences in a document, or turns in a dialogue.

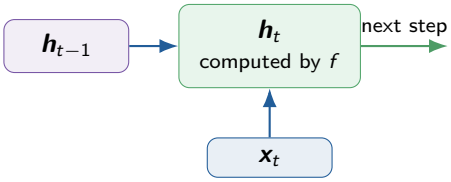
The memory idea

At position t, a recurrent model maintains a **hidden state**:

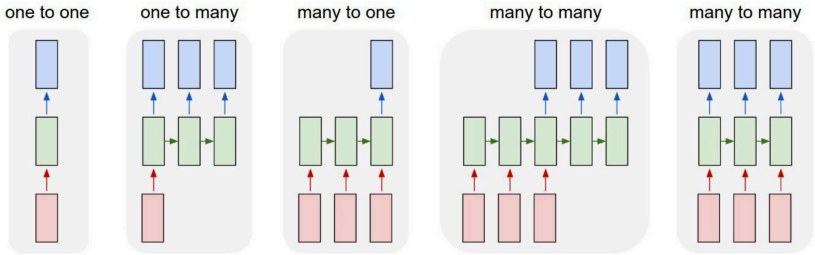
$$\boldsymbol{h}_t = f(\boldsymbol{x}_t, \boldsymbol{h}_{t-1})$$

- $\boldsymbol{x}_t$ contributes new information;
- $\boldsymbol{h}_{t-1}$ summarises earlier positions;
- $\boldsymbol{h}_t$ becomes the context for the next step.

The state is a learned vector, not a literal copy of all previous words.



Sequence input–output patterns



NLP examples

- one-to-many: caption generation;
- many-to-one: sentiment classification;
- aligned many-to-many: POS tagging or NER.

Different-length sequences

Machine translation and summarisation map a source sequence to a target sequence, usually with an encoder-decoder architecture.

Figure source: CS231n, *Recurrent Neural Networks*

Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

A recurrent neural network cell

An RNN applies the same parameterised recurrence at every position:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}),$$
$$\theta = \{W_{xh}, W_{hh}, \boldsymbol{b}_h\}.$$

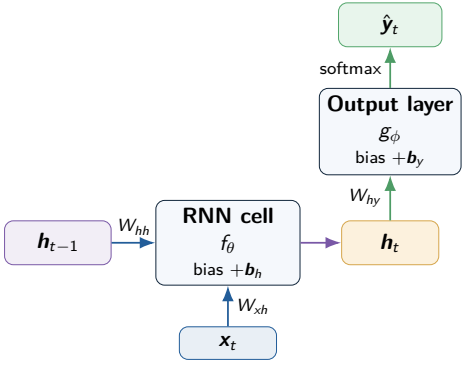
Here, θ collectively denotes all parameters inside the recurrent function f_θ.

Vanilla RNN state update

$$\boldsymbol{h}_t = \tanh(W_{xh}\boldsymbol{x}_t + W_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h)$$

Optional output layer

$$\boldsymbol{o}_t = g_\phi(\boldsymbol{h}_t) = W_{hy}\boldsymbol{h}_t + \boldsymbol{b}_y,$$
$$\hat{\boldsymbol{y}}_t = \mathrm{softmax}(\boldsymbol{o}_t),$$
$$\phi = \{W_{hy}, \boldsymbol{b}_y\}.$$

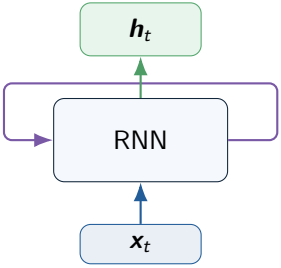


θ: recurrent-cell parameters

ϕ: output-layer parameters

Folded and unrolled views

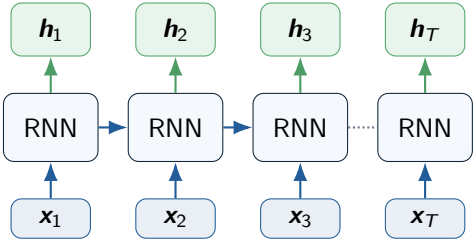
Folded view



for $t = 1, 2, \ldots, T$

One cell is reused.

Unrolled through positions

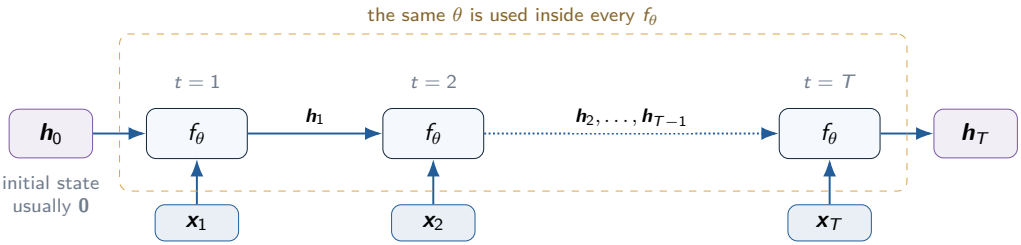


same parameters at every position

Why unroll?

The unrolled view exposes the sequence of computations used in the forward pass and in backpropagation through time.

Parameter sharing across time: unrolled view



Position-dependent inputs and states

inputs: $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T$

states: $\boldsymbol{h}_0, \boldsymbol{h}_1, \ldots, \boldsymbol{h}_T$

Parameters shared across all positions

$$\theta = \{W_{xh}, W_{hh}, \boldsymbol{b}_h\}$$

The model does not learn separate

$$\theta_1, \theta_2, \ldots, \theta_T.$$

Forward-pass

1. Initialise the hidden state

$$\boldsymbol{h}_0 = \mathbf{0} \quad \text{or a learned vector.}$$

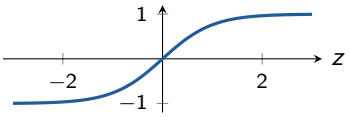
2. Repeat for $t = 1, \ldots, T$

$$\boldsymbol{a}_t = W_{xh}\boldsymbol{x}_t + W_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h,$$

$$\boldsymbol{h}_t = \tanh(\boldsymbol{a}_t),$$

$$\boldsymbol{o}_t = W_{hy}\boldsymbol{h}_t + \boldsymbol{b}_y.$$

Hyperbolic tangent: $\tanh(z) \in (-1, 1)$



Why computation is sequential

To compute $\boldsymbol{h}_t$, the RNN needs the previous state $\boldsymbol{h}_{t-1}$. Therefore, the hidden states within one sequence must be computed in order:

$$\boldsymbol{h}_0 \to \boldsymbol{h}_1 \to \boldsymbol{h}_2 \to \cdots \to \boldsymbol{h}_T.$$

Choosing the output

- **Classification:** use the final state $\boldsymbol{h}_T$;
- **Tagging:** use every state $\boldsymbol{h}_t$;
- **Language modelling:** make a prediction after every $\boldsymbol{h}_t$.

Worked example: a scalar RNN

Let the parameters be

$$W_{xh} = 0.5, \qquad W_{hh} = 0.8, \qquad b_h = 0.$$

Using one-dimensional inputs and hidden states, the recurrence becomes

$$h_t = \tanh(0.5x_t + 0.8h_{t-1}), \qquad h_0 = 0.$$

For the input sequence

$$(x_1, x_2, x_3) = (1,\ 0.5,\ -1),$$

we will compute h_1, h_2, and h_3 sequentially.

Interpretation

Before applying tanh, each new state combines:

- the current input x_t, weighted by 0.5;
- the previous state h_{t-1}, weighted by 0.8.

Thus, information from an earlier position can influence the states at later positions.

Hyperbolic tangent function

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

It transforms any real-valued input into the range $-1 < \tanh(z) < 1$.
In particular, $\tanh(0) = 0$; while large positive and negative values approach 1 and -1, respectively.

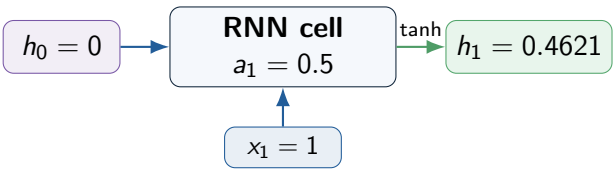
Worked example: step $t = 1$

Given: $W_{xh} = 0.5, \quad W_{hh} = 0.8, \quad b_h = 0, \quad h_0 = 0, \ (x_1, x_2, x_3) = (1, 0.5, -1).$

At the first position, there is no earlier information, so the state depends only on x_1.

$$\begin{aligned} a_1 &= 0.5x_1 + 0.8h_0 \\ &= 0.5(1) + 0.8(0) \\ &= 0.5, \end{aligned}$$

$$h_1 = \tanh(0.5) = 0.4621.$$

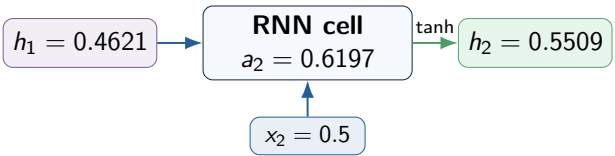


Worked example: step $t = 2$

Given: $W_{xh} = 0.5, \quad W_{hh} = 0.8, \quad b_h = 0, \quad h_0 = 0, \, (x_1, x_2, x_3) = (1, 0.5, -1).$

The state now depends on both x_2 and information carried from x_1 through h_1.

$$
\begin{aligned}
a_2 &= 0.5x_2 + 0.8h_1 \\
&= 0.5(0.5) + 0.8(0.4621) \\
&= 0.6197,
\end{aligned}
$$



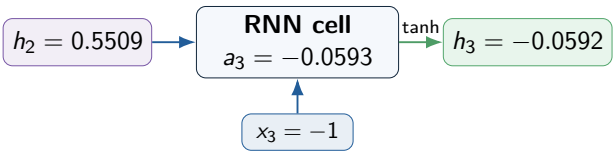
$$h_2 = \tanh(0.6197) = 0.5509.$$

Worked example: step $t = 3$

Given: $W_{xh} = 0.5, \quad W_{hh} = 0.8, \quad b_h = 0, \quad h_0 = 0, \ (x_1, x_2, x_3) = (1, 0.5, -1).$

The state now depends on x_3 and information carried from earlier inputs through h_2.

$$\begin{aligned} a_3 &= 0.5x_3 + 0.8h_2 \\ &= 0.5(-1) + 0.8(0.5509) \\ &= -0.0593, \end{aligned}$$



$$h_3 = \tanh(-0.0593) = -0.0592.$$

Worked example: what the recurrence did

t	x_t	h_{t-1}	$a_t = 0.5x_t + 0.8h_{t-1}$	$h_t = \tanh(a_t)$
1	1.0	0.0000	0.5000	0.4621
2	0.5	0.4621	0.6197	0.5509
3	-1.0	0.5509	-0.0593	-0.0592

- **Current evidence:** the term $0.5x_t$ allows the current input to update the state.
- **Carried context:** the term $0.8h_{t-1}$ allows information from earlier inputs to influence later states.

General lesson: At each position t, the hidden state is updated to form a new fixed-dimensional representation of the sequence prefix $(x_1, \ldots, x_t)$.

Worked example: from hidden state to prediction

After processing the entire sequence, the RNN produces the final hidden state h_3.

1. Compute the output logit

For binary sentiment classification, the output layer converts h_3 into a scalar logit:

$$o_3 = W_{hy} h_3 + b_y,$$

where the output-layer parameters are

$$W_{hy} = 1.2, \qquad b_y = 0.1.$$

Using $h_3 \approx -0.0592$:

$$\begin{aligned} o_3 &= 1.2(-0.0592) + 0.1 \\ &= 0.02896 \end{aligned}$$

2. Convert the logit into a probability

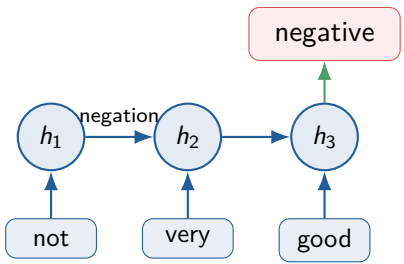
Apply the sigmoid function to obtain the probability of class 1:

$$\begin{aligned} p(y = 1 \mid x_{1:3}) &= \text{sigmoid}(o_3) \\ &= \frac{1}{1 + e^{-0.02896}} \\ &\approx 0.5072. \end{aligned}$$

What can the hidden state remember?

During training, the RNN learns hidden-state representations that can encode task-relevant information, such as:

- whether a negation has appeared;
- whether the subject is singular or plural;
- which topic or entity is currently active;
- how positive or negative the prefix is;
- what kinds of words are likely to follow.



Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

One-to-many: conditional text generation

Given a single conditioning vector $\boldsymbol{x}$, the first hidden state is

$$\boldsymbol{h}_1 = f_\theta(\boldsymbol{x}, \boldsymbol{h}_0).$$

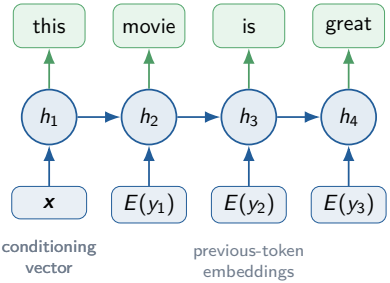
At each position, the hidden state produces a token distribution:

$$\hat{\boldsymbol{y}}_t = \text{softmax}\left(W_{hy}\boldsymbol{h}_t + \boldsymbol{b}_y\right).$$

During generation, the selected token y_t is embedded and supplied to the next recurrent step:

$$\boldsymbol{h}_{t+1} = f_\theta\left(E(y_t), \boldsymbol{h}_t\right).$$

- the conditioning vector starts the generation process;
- each hidden state predicts one token;
- the selected token becomes the next input;



Example output:

this movie is great

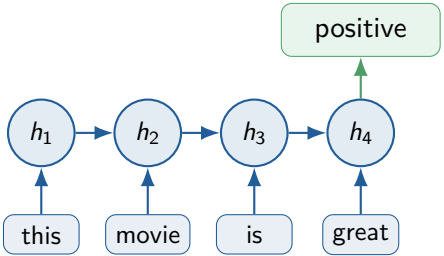
Many-to-one: sentence classification

For sentiment classification:

$$\boldsymbol{h}_t = \mathrm{RNN}(\boldsymbol{x}_t, \boldsymbol{h}_{t-1})$$

$$\hat{\boldsymbol{y}} = \mathrm{softmax}(W_{hy}\boldsymbol{h}_T + \boldsymbol{b}_y).$$

- the RNN reads the sentence left to right;
- $\boldsymbol{h}_T$ summarises the full prefix;
- the output layer maps the final hidden state to class probabilities.

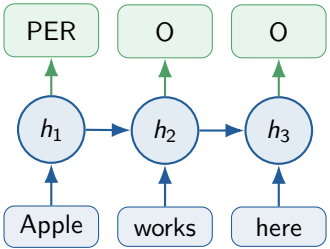


Many-to-many: sequence labelling

For POS tagging or named entity recognition:

$$\hat{\boldsymbol{y}}_t = \text{softmax}(W_{hy}\boldsymbol{h}_t + \boldsymbol{b}_y), \qquad t = 1, \ldots, T.$$

- input and output lengths are aligned;
- each token receives a label;
- the state supplies left-context information;
- one output-layer parameter set is shared.



Limitation of a left-to-right RNN

When predicting the label for *Apple*, it has not yet seen *works here*. Bidirectional models address this when future context is available.

RNN Language Modelling: Prediction and Training

Next-token prediction
At each step, the RNN uses the previous token and its hidden state to predict the next token.

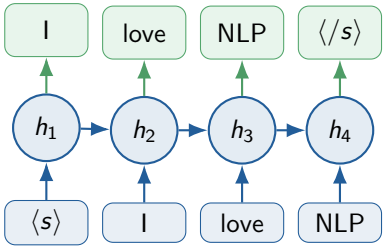
Teacher forcing
During training, the actual previous token is provided as input, and the model learns to predict the correct next token.

Training objective
Minimise the average negative log-likelihood:

$$\mathcal{L}_t = -\log p(w_t \mid w_{0:t-1}),$$

$$\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T} \mathcal{L}_t.$$

Perplexity, $\mathrm{PPL} = \exp(\mathcal{L})$
Lower perplexity indicates better next-token prediction on average.



Shifted prediction: Each input token is used to predict the next token.

Green boxes represent the target tokens during training.

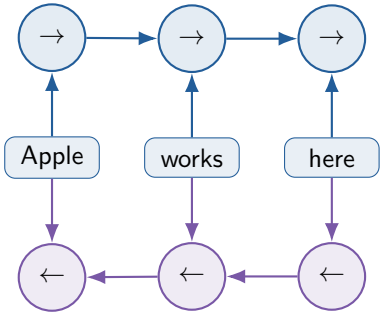
Bidirectional RNN

A bidirectional RNN computes:

$$\overrightarrow{\boldsymbol{h}}_t = \overrightarrow{\mathrm{RNN}}\left(\boldsymbol{x}_t, \overrightarrow{\boldsymbol{h}}_{t-1}\right),$$

$$\overleftarrow{\boldsymbol{h}}_t = \overleftarrow{\mathrm{RNN}}\left(\boldsymbol{x}_t, \overleftarrow{\boldsymbol{h}}_{t+1}\right),$$

$$\boldsymbol{h}_t^{\mathrm{bi}} = \left[\overrightarrow{\boldsymbol{h}}_t; \overleftarrow{\boldsymbol{h}}_t\right].$$



Each token representation uses both left and right context.

Bidirectional RNNs are suitable when the complete sequence is available, such as named-entity recognition and part-of-speech tagging. They are unsuitable for real-time next-token generation because the future tokens required by the backward RNN do not yet exist.

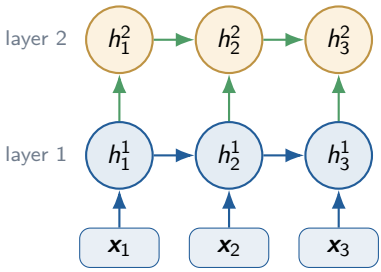
Stacked RNNs

Multiple recurrent layers can be stacked:

$$\boldsymbol{h}_t^{(1)} = \mathrm{RNN}^{(1)}(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}^{(1)})$$

$$\boldsymbol{h}_t^{(2)} = \mathrm{RNN}^{(2)}(\boldsymbol{h}_t^{(1)}, \boldsymbol{h}_{t-1}^{(2)}).$$

- horizontal connections carry information through time;
- vertical connections pass representations between layers;
- depth increases capacity and optimisation difficulty.



Variable lengths: padding and masking

Sequences in a mini-batch are often padded to a common length:

```
[I, like, NLP, PAD, PAD]
[This, model, is, very, useful]
```

A binary mask identifies real tokens:

$$[1, 1, 1, 0, 0]$$
$$[1, 1, 1, 1, 1]$$

Use the mask to:

- exclude padding from the loss;
- ignore padded output positions;
- select the last real hidden state;
- avoid misleading evaluation statistics.

For token-level tasks, the masked loss can be expressed as

$$\mathcal{L} = \frac{\sum_i \sum_t m_{i,t} \mathcal{L}_{i,t}}{\sum_i \sum_t m_{i,t}},$$

where

$$m_{i,t} = \begin{cases} 1, & \text{if position } t \text{ contains a real token,} \\ 0, & \text{if position } t \text{ contains padding.} \end{cases}$$

Thus, positions with $m_{i,t} = 0$ do not contribute to the loss.

Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

Loss may occur at one or many positions

Sequence classification

$$\mathcal{L} = \mathrm{CE}(\boldsymbol{y}, \hat{\boldsymbol{y}})$$

One loss is attached to the sequence representation.

Tagging or language modelling

$$\mathcal{L} = \frac{\sum_{t=1}^{T} m_t \, \mathrm{CE}(\boldsymbol{y}_t, \hat{\boldsymbol{y}}_t)}{\sum_{t=1}^{T} m_t}$$

The mask m_t excludes padded positions.

Why the losses are coupled

The loss $\mathcal{L}_t$ at position t is local to the prediction $\hat{\boldsymbol{y}}_t$, but it is not independent of earlier computations:

$$\boldsymbol{h}_t = f_\theta\left(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}\right).$$

Because $\boldsymbol{h}_t$ depends on earlier hidden states, $\mathcal{L}_t$ can propagate gradients backwards to earlier positions. Furthermore, the same parameters θ are reused at every position:

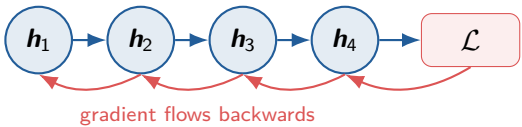
$$\nabla_\theta \mathcal{L} = \frac{\sum_{t=1}^{T} m_t \nabla_\theta \mathcal{L}_t}{\sum_{t=1}^{T} m_t}, \qquad \mathcal{L}_t = \mathrm{CE}(\boldsymbol{y}_t, \hat{\boldsymbol{y}}_t).$$

Therefore, all valid token-level losses jointly update the same recurrent parameters.

Backpropagation through time: the idea

Backpropagation through time (BPTT):

1. unroll the RNN over the sequence;
2. run the forward pass and store intermediate states;
3. propagate gradients from later computations to earlier ones;
4. sum gradient contributions for each shared parameter;
5. update the shared parameters.



Same principle as ordinary backpropagation

BPTT applies the chain rule to an unrolled computational graph. The phrase *through time* describes the repeated recurrent steps, not a different optimiser.

One parameter receives gradients from many positions

Although the unrolled RNN shows a computation at every position, it does not contain a different W_{hh} at each step. The same matrix is reused:

$$\boldsymbol{h}_t = \tanh(W_{xh}\boldsymbol{x}_t + W_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h)\,.$$

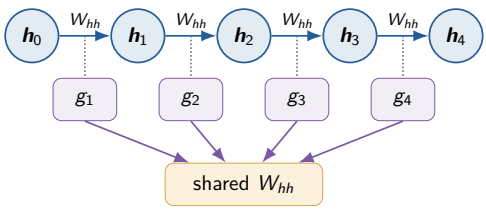
Each use of W_{hh} contributes a gradient:

$$g_t = \left.\frac{\partial \mathcal{L}}{\partial W_{hh}}\right|_{\text{use at step } t}.$$

BPTT adds the contributions from all positions:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = g_1 + g_2 + \cdots + g_T.$$

The optimiser then performs one update to the shared matrix W_{hh}.



one accumulated gradient and one parameter update

An automatic differentiation system, such as PyTorch autograd, records the operations performed during the forward pass and stores values needed for backpropagation. During BPTT, it traverses these operations backwards and accumulates the gradient contributions for each shared parameter.

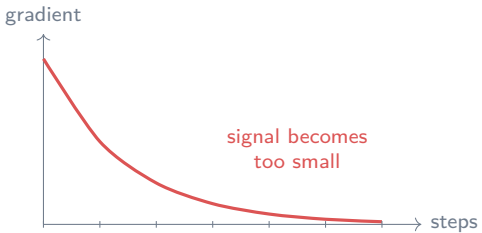
Vanishing Gradients

Suppose a gradient is multiplied by 0.5 at each step:

$$1,\ 0.5,\ 0.25,\ 0.125, \ldots$$

$$0.5^{20} \approx 9.54 \times 10^{-7}.$$

- early states receive almost no learning signal;
- long-distance dependencies are hard to learn;
- tanh saturation can make the effect worse: when the pre-activation is large, tanh is close to 1 or -1, and its derivative $1 - \tanh^2(a_t)$ is close to zero.



Note

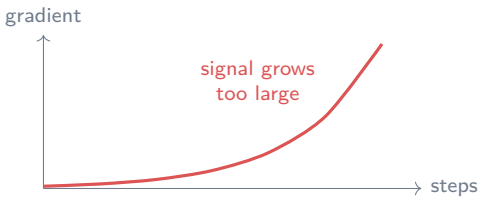
Vanishing gradients prevent the model from learning which much earlier inputs were important for a later prediction.

Exploding Gradients

Suppose a gradient is multiplied by 1.5 at each step:

$$1,\ 1.5,\ 2.25,\ 3.375, \ldots$$

$$1.5^{20} \approx 3.33 \times 10^{3}.$$



- updates can become extremely large;
- loss may oscillate or become NaN;
- training becomes numerically unstable.

Note

Exploding gradients cause unstable parameter updates.

Gradient clipping

Let $\boldsymbol{g}$ contain the gradients of all model parameters. Global-norm clipping rescales $\boldsymbol{g}$ when its norm exceeds a threshold τ:

$$\boldsymbol{g}_{\text{clip}} = \begin{cases} \boldsymbol{g}, & \|\boldsymbol{g}\|_2 \le \tau, \\ \dfrac{\tau}{\|\boldsymbol{g}\|_2}\boldsymbol{g}, & \|\boldsymbol{g}\|_2 > \tau. \end{cases}$$

Example:

$$\|\boldsymbol{g}\|_2 = 20 \qquad \text{and} \qquad \tau = 5,$$

then every gradient component is multiplied by

$$\frac{\tau}{\|\boldsymbol{g}\|_2} = \frac{5}{20} = 0.25.$$

Therefore,

$$\boldsymbol{g}_{\text{clip}} = 0.25\boldsymbol{g}, \qquad \|\boldsymbol{g}_{\text{clip}}\|_2 = 5.$$

The gradient direction is preserved, but its magnitude is limited.

Limitation: Gradient clipping controls exploding gradients, but it cannot restore a learning signal that has already vanished.

The problem of long-term dependencies

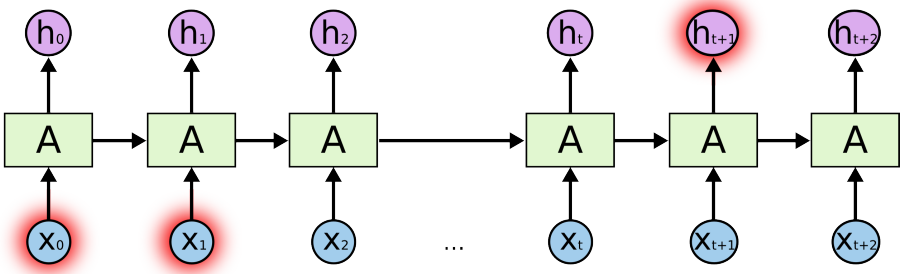


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

Example:

I grew up in France . . . I speak fluent French,

predicting *French* requires the model to remember information about *France*, which appeared many positions earlier.

Key challenge: A vanilla RNN must carry relevant information through many repeated state updates. Over a long sequence, earlier information may become weakened or overwritten.

Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

Why add gates?

A vanilla RNN has difficulty learning long-range dependencies. It rewrites its entire hidden state at every step:

$$\boldsymbol{h}_t = \tanh(W_{xh}\boldsymbol{x}_t + W_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h).$$

For long sequences, it would help to learn:

- what old information to keep;
- what old information to remove;
- what new information to write;
- what part of memory to expose.

LSTM design

A Long Short-Term Memory (LSTM) introduces a cell state and multiplicative gates that learn these decisions.

From a vanilla RNN to an LSTM

An RNN can be viewed as a chain that applies the same recurrent module at every time step. In a vanilla RNN, this module is relatively simple, typically consisting of a single tanh transformation.

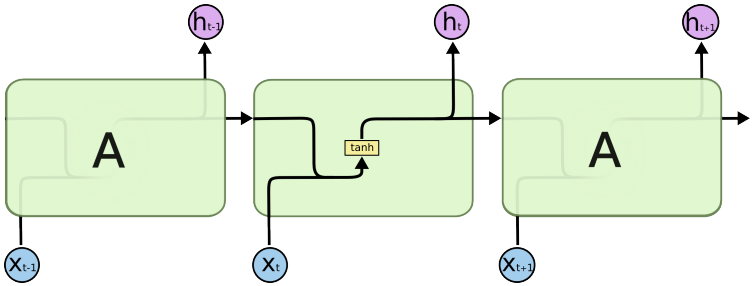


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

From a vanilla RNN to an LSTM

LSTMs retain this chain-like organisation but use a more sophisticated recurrent module. Instead of a single transformation, each module contains four interacting components that control how information is retained, updated and output.

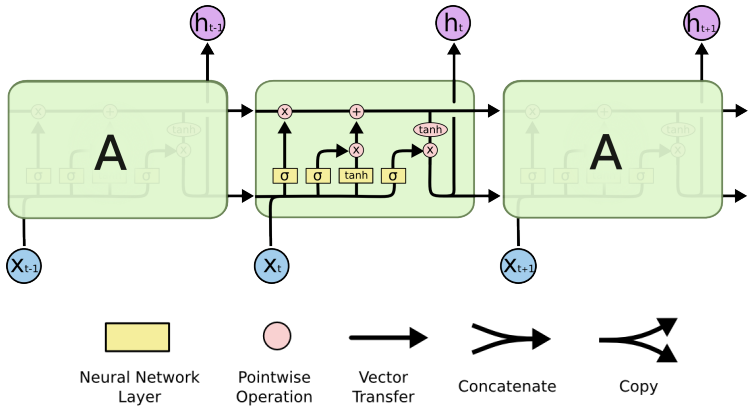


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

Core idea: the cell-state

The key feature of an LSTM is its cell state $\boldsymbol{c}_t$, represented by the horizontal path across the top of the diagram. It provides a direct memory pathway through the sequence, allowing information to be preserved across many time steps. The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called **gates**.

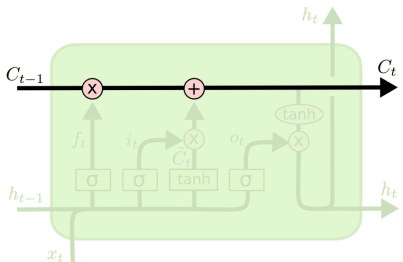


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

What is a gate?

A gate is a learned **soft switch** that controls how much information is allowed to pass.

1. Compute the gate values

$$\boldsymbol{g}_t = \text{sigmoid}(W_g \boldsymbol{x}_t + U_g \boldsymbol{h}_{t-1} + \boldsymbol{b}_g), \qquad 0 < g_{t,j} < 1.$$

The current input and previous hidden state determine a separate control value for each vector component.

2. Apply the gate

For an information vector $\boldsymbol{v}_t$,

$$\boldsymbol{v}_t^{\text{gated}} = \boldsymbol{g}_t \odot \boldsymbol{v}_t,$$

where $\odot$ denotes element-wise multiplication.

- $g_{t,j} \approx 0$: almost block component j;
- $g_{t,j} \approx 1$: pass most of component j;
- $0 < g_{t,j} < 1$: pass it partially.

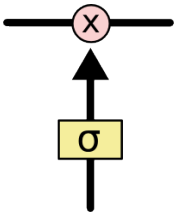


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

A vector gate makes a separate soft decision for each dimension. It acts like a collection of learned dimmer switches rather than a single on/off switch.

Step 1: forget gate

The forget gate decides how much of the previous cell state should be retained:

$$\boldsymbol{f}_t = \text{sigmoid}(W_f \boldsymbol{x}_t + U_f \boldsymbol{h}_{t-1} + \boldsymbol{b}_f)\,.$$

Each component satisfies

$$0 < f_{t,j} < 1, \qquad j = 1, \ldots, d_h.$$

The gate scales the previous cell state through element-wise multiplication:

$$\boldsymbol{f}_t \odot \boldsymbol{c}_{t-1}.$$

- $f_{t,j} \approx 1$: retain memory component j;
- $f_{t,j} \approx 0$: remove memory component j;
- intermediate values: retain it partially.

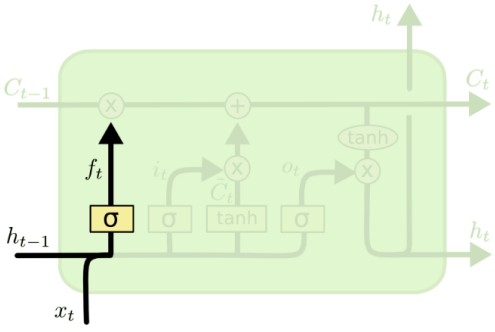


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

Step 2: input gate and candidate content

The input gate decides how much of each candidate component should be added to the cell state:

$$\boldsymbol{i}_t = \text{sigmoid}(W_i \boldsymbol{x}_t + U_i \boldsymbol{h}_{t-1} + \boldsymbol{b}_i)\,.$$

A vector of candidate cell-state values, $\widetilde{\boldsymbol{c}}_t$, is computed as

$$\widetilde{\boldsymbol{c}}_t = \tanh(W_c \boldsymbol{x}_t + U_c \boldsymbol{h}_{t-1} + \boldsymbol{b}_c)\,.$$

The gated candidate content to be updated is

$$\boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t.$$

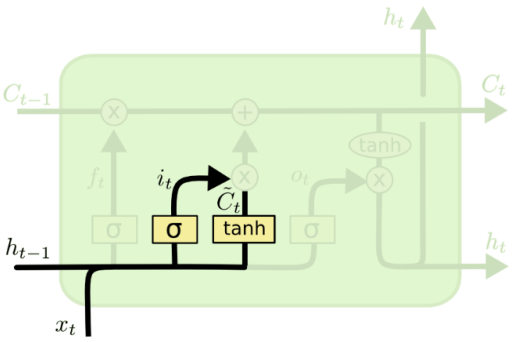


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

Interpretation: The candidate determines *what could be stored*, while the input gate determines *how much of each component should be stored*.

Step 3: update the cell state

The new cell state combines two sources of information:

- **Retained old memory:** $\boldsymbol{f}_t \odot \boldsymbol{c}_{t-1}$;
- **Selected new content:** $\boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t$.

The two contributions are added to form the updated cell state:

$$\boldsymbol{c}_t = \underbrace{\boldsymbol{f}_t \odot \boldsymbol{c}_{t-1}}_{\text{retained old memory}} + \underbrace{\boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t}_{\text{selected new content}}$$

Thus, the forget gate controls what is retained, while the input gate controls what new information is added.

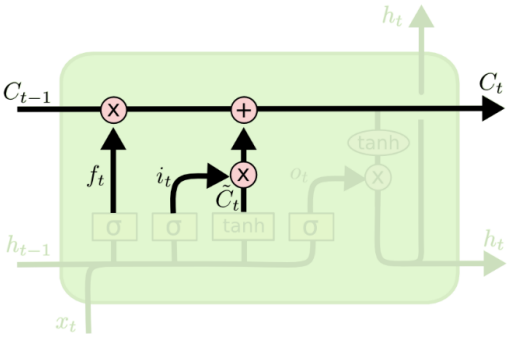


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

Why does this help with vanishing gradients?

When the forget gate is close to 1, gradients can propagate more effectively along the direct cell-state pathway, helping the LSTM learn longer-range dependencies.

Step 4: output gate and hidden state

The output gate controls how much each component of the updated cell state contributes to the hidden state:

$$\boldsymbol{o}_t = \text{sigmoid}(W_o \boldsymbol{x}_t + U_o \boldsymbol{h}_{t-1} + \boldsymbol{b}_o)\,.$$

The updated cell state is transformed by tanh and then modulated element-wise by the output gate:

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t).$$

- $\boldsymbol{c}_t$: the updated cell state;
- $\boldsymbol{o}_t$: the output gate;
- $\boldsymbol{h}_t$: the hidden state passed to the next time step and the output layer.

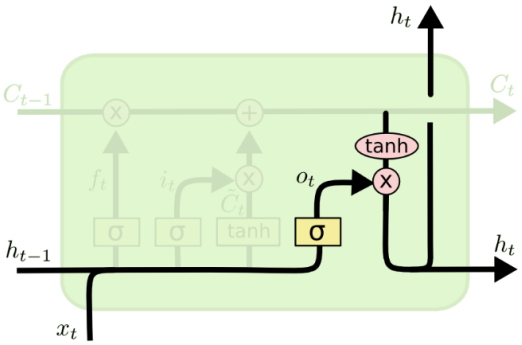


Figure source: Christopher Olah, *Understanding LSTM Networks* (2015)

Interpretation: The output gate controls the contribution of the cell state to the current hidden state. Information can remain in $\boldsymbol{c}_t$ even when its contribution to $\boldsymbol{h}_t$ is small.

Complete LSTM cell

At position t, the LSTM uses the current input $\boldsymbol{x}_t$, the previous hidden state $\boldsymbol{h}_{t-1}$, and the previous cell state $\boldsymbol{c}_{t-1}$.

Forget gate

$$\boldsymbol{f}_t = \text{sigmoid}(W_f \boldsymbol{x}_t + U_f \boldsymbol{h}_{t-1} + \boldsymbol{b}_f)\,.$$

Input gate and candidate

$$\boldsymbol{i}_t = \text{sigmoid}(W_i \boldsymbol{x}_t + U_i \boldsymbol{h}_{t-1} + \boldsymbol{b}_i)\,,$$

$$\widetilde{\boldsymbol{c}}_t = \tanh(W_c \boldsymbol{x}_t + U_c \boldsymbol{h}_{t-1} + \boldsymbol{b}_c)\,.$$

Cell-state update

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t.$$

Output gate and hidden state

$$\boldsymbol{o}_t = \text{sigmoid}(W_o \boldsymbol{x}_t + U_o \boldsymbol{h}_{t-1} + \boldsymbol{b}_o)\,,$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t).$$

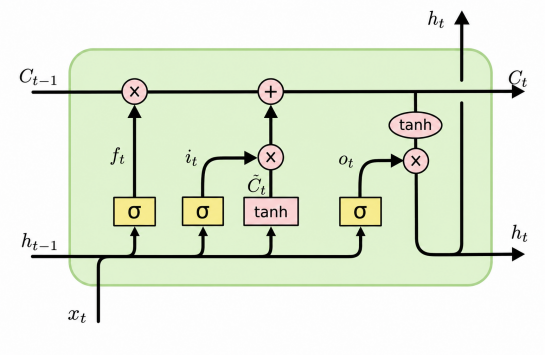


Figure source: Diagram adapted from Christopher Olah, *Understanding LSTM Networks* (2015)

The forget, input, cell-update, and output pathways operate together at every time step.

Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

Gated Recurrent Units (GRU)

A gated recurrent unit (GRU) provides a simpler gated alternative to an LSTM. It maintains one recurrent state, $\boldsymbol{h}_t$, rather than separate cell and hidden states.

- the **reset gate** $\boldsymbol{r}_t$ controls how the previous state contributes to the candidate state;
- the **update gate** $\boldsymbol{z}_t$ controls how the previous state and candidate state are combined;
- the resulting state $\boldsymbol{h}_t$ is passed to the next time step or the output layer.

Unlike an LSTM, a GRU has no separate cell state $\boldsymbol{c}_t$ and no output gate.

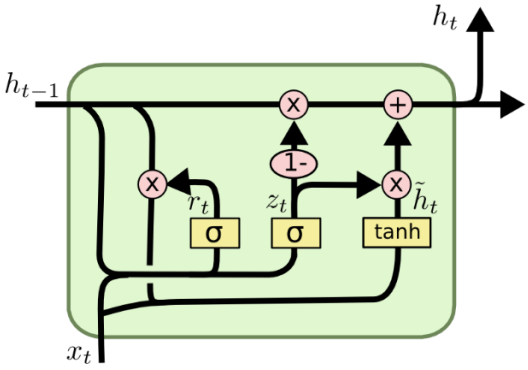


Figure source: Diagram adapted from Christopher Olah, *Understanding LSTM Networks* (2015)

Step 1: reset gate

The reset gate controls how much of the previous hidden state contributes to the candidate state:

$$\boldsymbol{r}_t = \text{sigmoid}(W_r \boldsymbol{x}_t + U_r \boldsymbol{h}_{t-1} + \boldsymbol{b}_r)$$

It is applied to the previous state elementwise:

$$\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}.$$

Because $0 < r_{t,j} < 1$:

- $r_{t,j} \approx 0$: use little of previous component j;
- $r_{t,j} \approx 1$: use most of previous component j.

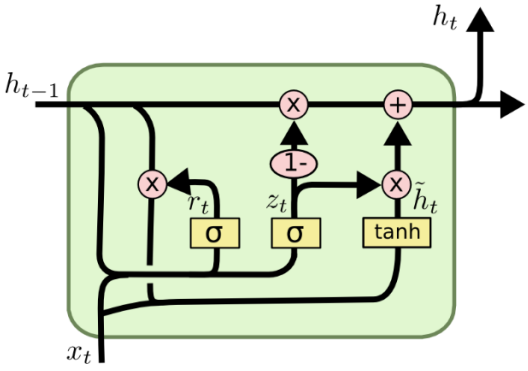


Figure source: Diagram adapted from Christopher Olah, *Understanding LSTM Networks* (2015)

The reset gate does not directly remove information from the final state. It only controls how previous information contributes to the candidate state.

Step 2: update gate

The update gate controls the balance between the previous state and the candidate state:

$$\boldsymbol{z}_t = \text{sigmoid}(W_z \boldsymbol{x}_t + U_z \boldsymbol{h}_{t-1} + \boldsymbol{b}_z)$$

Because $0 < z_{t,j} < 1$:

- $z_{t,j} \approx 0$: preserve previous component $h_{t-1,j}$;
- $z_{t,j} \approx 1$: select candidate component $\widetilde{h}_{t,j}$;
- intermediate values: combine the two components.

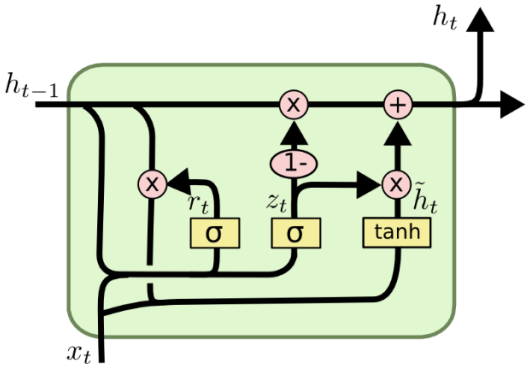


Figure source: Diagram adapted from Christopher Olah, *Understanding LSTM Networks* (2015)

Unlike the reset gate, the update gate directly controls the final hidden-state update.

Step 3: candidate hidden state

The candidate state proposes new hidden-state content:

$$\widetilde{\boldsymbol{h}}_t = \tanh(W_h \boldsymbol{x}_t + U_h(\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}) + \boldsymbol{b}_h)\,.$$

It combines:

- the current input through $W_h \boldsymbol{x}_t$;
- reset-controlled previous information through $U_h(\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1})$;
- a tanh transformation that bounds each candidate component between -1 and 1

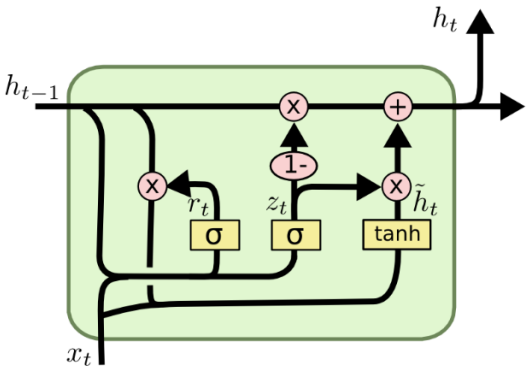


Figure source: Diagram adapted from Christopher Olah, *Understanding LSTM Networks* (2015)

The candidate $\widetilde{\boldsymbol{h}}_t$ is a proposed new representation; the update gate determines how much of it contributes to the new state.

Step 4: update the hidden state

The new hidden state is a component-wise combination of the previous state and candidate state:

$$\boldsymbol{h}_t = \underbrace{(1 - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1}}_{\text{preserved previous state}} + \underbrace{\boldsymbol{z}_t \odot \widetilde{\boldsymbol{h}}_t}_{\text{selected candidate state}}$$

Because $0 < z_{t,j} < 1$, the update gate provides a learned interpolation between retaining previous information and incorporating candidate content.

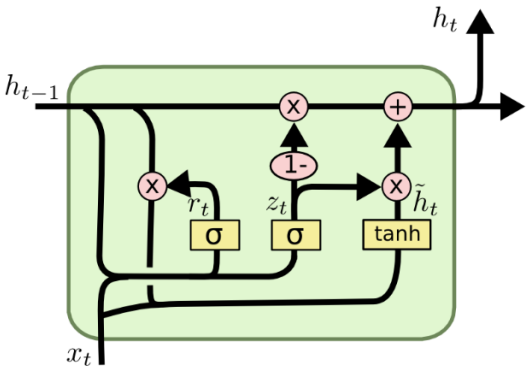


Figure source: Diagram adapted from Christopher Olah, *Understanding LSTM Networks* (2015)

Different hidden-state components can be preserved or updated by different amounts at the same time step.

Complete GRU cell

At position t, the GRU uses the current input $\boldsymbol{x}_t$ and the previous hidden state $\boldsymbol{h}_{t-1}$.

Reset gate

$$\boldsymbol{r}_t = \text{sigmoid}(W_r \boldsymbol{x}_t + U_r \boldsymbol{h}_{t-1} + \boldsymbol{b}_r).$$

Update gate

$$\boldsymbol{z}_t = \text{sigmoid}(W_z \boldsymbol{x}_t + U_z \boldsymbol{h}_{t-1} + \boldsymbol{b}_z).$$

Candidate hidden state

$$\widetilde{\boldsymbol{h}}_t = \tanh(W_h \boldsymbol{x}_t + U_h(\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}) + \boldsymbol{b}_h).$$

Hidden-state update

$$\boldsymbol{h}_t = (1 - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \widetilde{\boldsymbol{h}}_t.$$

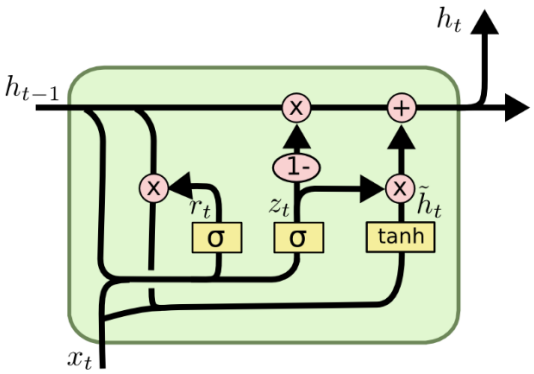


Figure source: Diagram adapted from Christopher Olah, *Understanding LSTM Networks* (2015)

The reset, candidate-construction, and update pathways operate together at every time step.

Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

Vanilla RNN, GRU and LSTM at a glance

Property	Vanilla RNN	GRU	LSTM
recurrent states	$\boldsymbol{h}_t$	$\boldsymbol{h}_t$	$\boldsymbol{h}_t, \boldsymbol{c}_t$
sigmoid gates	none	reset, update	forget, input, output
parameter factor	$1\times$	$3\times$	$4\times$
long-range training	difficult	improved	improved

At equal d_x and d_h, gated cells use more recurrent parameters but provide learned paths for preserving and updating information. No cell guarantees perfect memory.

Outline

1. Why Sequence Models?
2. Vanilla Recurrent Neural Networks
3. RNN Architectures for NLP
4. Training Recurrent Networks
5. Long Short-Term Memory
6. Gated Recurrent Units
7. Comparison and Practical Use
8. Summary

Lecture summary

- Sequence models preserve order by updating a context-dependent hidden state at every position.
- A vanilla RNN shares one recurrent transition across time and supports classification, tagging and language modelling.
- BPTT applies ordinary backpropagation to the unrolled recurrent graph.
- Repeated gradient multiplications can cause vanishing or exploding gradients. Gradient clipping targets explosion.
- An LSTM uses forget, input and output gates plus a cell state to control memory.
- A GRU uses reset and update gates and maintains one recurrent state.
- GRUs and LSTMs mitigate long-range training difficulty, but careful masking, directionality, regularisation and evaluation remain essential.

References

- Christopher Olah. *Understanding LSTM Networks*, 2015.
- CS231n. *Recurrent Neural Networks*.
- Jeffrey L. Elman. Finding Structure in Time. *Cognitive Science*, 1990.
- Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky and Sanjeev Khudanpur. Recurrent Neural Network Based Language Model. *Interspeech*, 2010.
- Razvan Pascanu, Tomas Mikolov and Yoshua Bengio. On the Difficulty of Training Recurrent Neural Networks. *ICML*, 2013.
- Sepp Hochreiter and Juergen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 1997.
- Kyunghyun Cho et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *EMNLP*, 2014.
- Alex Graves. *Supervised Sequence Labelling with Recurrent Neural Networks*. Springer, 2012.
- Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- Daniel Jurafsky and James H. Martin. *Speech and Language Processing*, 3rd edition draft.